Security Assessment Report

Consolidated defensive assessment of an authorized lab environment (OWASP Juice Shop and local network traffic)

Prepared by	Mubassir, Security Intern, SWYNEX Technologies
Date	November 2026
Program	SWYNEX Technologies Internship, Task 4
Based on	Task 1 Security Fundamentals Assessment; Task 2 Network Security Analysis; Task 3 Web Security Review
Classification	Educational lab report. Credentials, tokens and personal data redacted.

1. Scope and Authorization

All testing was performed in an isolated lab that I own and control. The target was OWASP Juice Shop, an intentionally vulnerable training application, running in a Docker container on `localhost:3000` inside a Kali Linux virtual machine. Network analysis used traffic I generated myself, plus an Nmap scan of my own loopback interface. No third-party or production systems were accessed, scanned or attacked.

Component	Detail
Host	Windows with VMware Workstation
Guest OS	Kali Linux
Target	OWASP Juice Shop (bkimminich/juice-shop, Docker) at http://localhost:3000
Tools	Nmap, Nikto, curl, Burp Suite Community, Firefox DevTools, Wireshark, Python 3 with Scapy

Credentials, session tokens and personal identifiers have been removed or redacted from this report and its evidence.

2. Executive Summary

This report combines the results of three assessments of a lab web application and its network traffic. In total, **8 findings** were documented: **1 Critical, 3 High, 2 Medium, 1 Low and 1 Informational**.

The most serious issue is a SQL injection flaw in the login form that lets anyone sign in as the administrator without a password. Two further High issues expose confidential files through an unauthenticated `/ftp/` directory and transmit login credentials and cookies in cleartext over HTTP. A port scan was also clearly visible in network captures, showing how easily this kind of reconnaissance can be detected when traffic is monitored.

The application is intentionally vulnerable, so these results are expected. In a real environment the overall risk would be **Critical**, because an attacker could take over administrative access and read confidential data with little effort.

Top priorities

- Fix the SQL injection by using parameterized queries.
- Remove public access to /ftp/ and enforce authentication on all non-public files.
- Serve everything over HTTPS and add security headers.

3. Methodology

The work followed a standard defensive assessment workflow:

- **Reconnaissance:** Nmap (`nmap -sV -p 3000 127.0.0.1`), curl -I and Nikto identified the open port, HTTP headers and obvious misconfigurations.
- **Traffic interception:** Burp Suite Community acted as a proxy (127.0.0.1:8080) so raw requests and responses could be inspected.
- **Manual validation:** each suspected issue was reproduced by hand (authentication, input handling, access control, configuration).
- **Triage:** about 150 Nikto hits for backup and certificate files (such as /backup.tar and /database.jks) were checked manually and discarded as false positives, because the single-page app returns HTTP 200 for almost any path.

- **Network analysis:** Wireshark captured browsing, a failed Juice Shop login, a ping test and an Nmap SYN scan against my own loopback. The main capture held 2,607 packets over about 112 seconds. Key counts were cross-checked with Python and Scapy.
- **Rating and reporting:** each finding was mapped to the OWASP Top 10 where relevant, rated and given remediation guidance.

4. Risk Rating Method

Each finding is rated by combining **likelihood** (how easy it is to find and exploit) with **impact** (what an attacker gains), using five levels: Critical, High, Medium, Low and Informational. Ratings assume the system were a real, internet-facing deployment. Where earlier task write-ups used a different rating for the same issue, this report uses one consistent rating and explains the change.

5. Findings Summary

ID	Finding	Source task	OWASP category	Risk
F1	SQL injection: authentication bypass	1, 3	A03:2021 Injection	**Critical**
F2	Sensitive file exposure at /ftp/	1, 3	A01:2021 Broken Access Control	**High**
F3	Credentials and cookies sent in cleartext HTTP	2	A02:2021 Cryptographic Failures	**High**
F4	Detectable TCP SYN port scan pattern	2	Detection and monitoring	**High**
F5	Cross-site scripting (XSS) in search	1, 3	A03:2021 Injection	**Medium**
F6	Unencrypted DNS and WPAD lookup	2	A05:2021 Security Misconfiguration	**Medium**
F7	Security misconfiguration: permissive CORS, missing headers, information disclosure	1, 3	A05:2021 Security Misconfiguration	**Low**
F8	Local SMB and IPv6 neighbour discovery traffic	2	Attack surface review	**Informational**

6. Detailed Findings

F1: SQL Injection, Authentication Bypass - Critical

Affected asset: `POST /rest/user/login` (email field)

Description: The login form places user input into a database query without sanitizing it. Entering `' or 1=1--` in the email field with any password logs the tester in as the administrator.

Impact: Complete authentication bypass and full administrative access with no valid credentials.

Evidence: Login form with payload; challenge-solved confirmation; Burp Suite capture of the login request and response, which returned a valid session token for the admin account (token redacted).

Rating note: Task 1 rated this High and Task 3 rated it Critical. It is rated Critical here because it grants full admin control with no skill or prior access required.

Remediation:

- Use parameterized queries or prepared statements.
- Validate authentication input on the server side.
- Use an ORM that parameterizes queries by default.
- Add rate limiting and alerting for repeated failed or malformed login attempts.

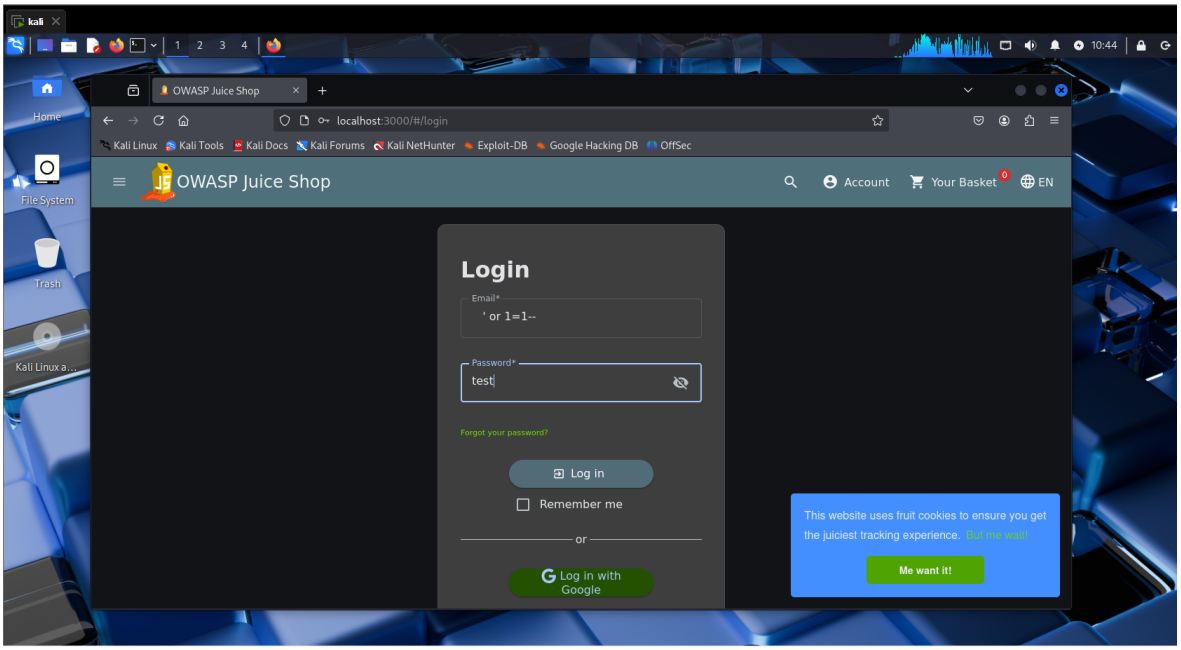


Figure F1-1: SQL injection payload entered in the login form

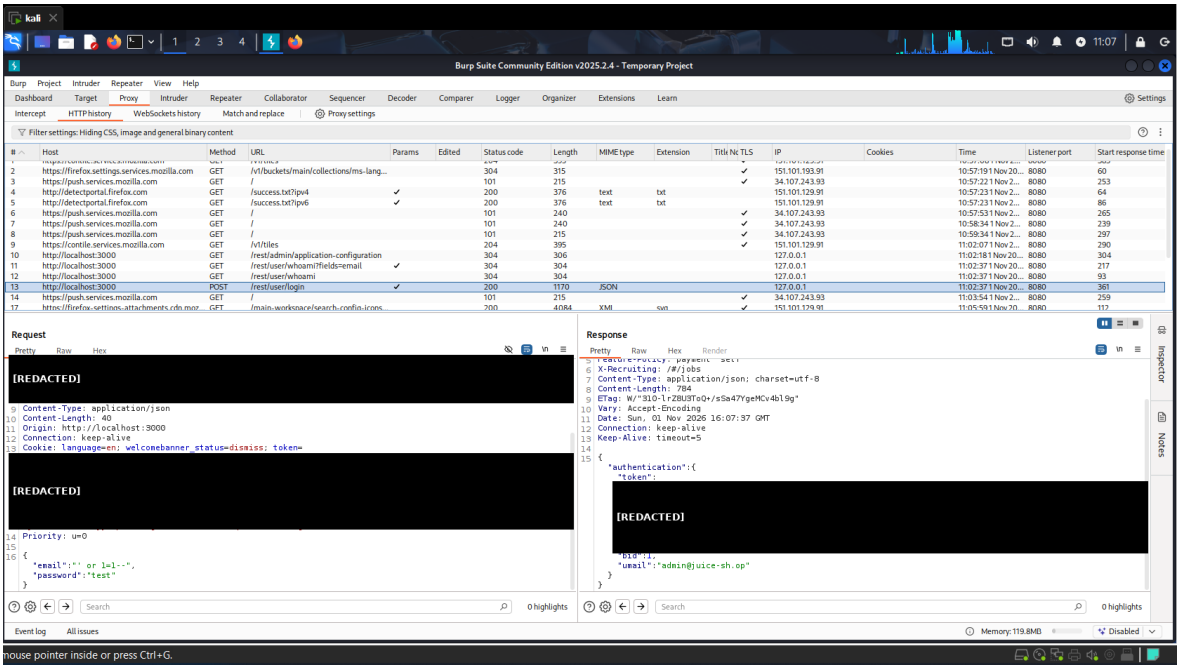


Figure F1-2: Burp Suite capture of POST /rest/user/login (tokens redacted)

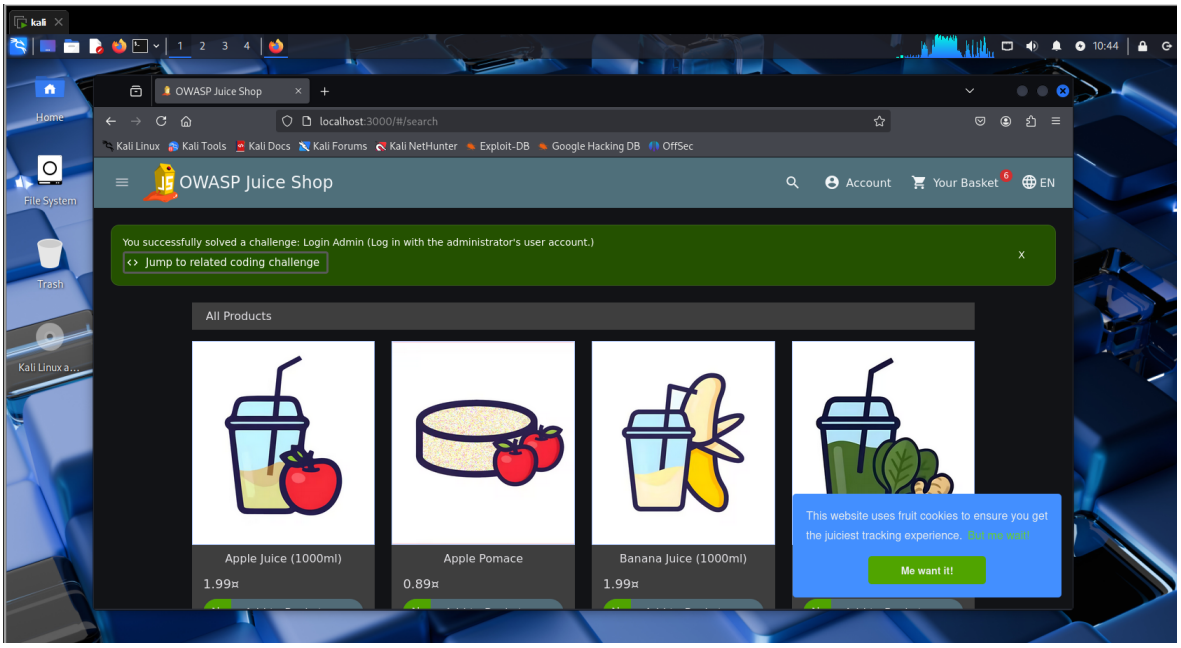


Figure F1-3: Login Admin challenge solved

F2: Sensitive File Exposure at /ftp/ - High

Affected asset: `GET /ftp/`

Description: The /ftp/ directory is reachable without authentication and shows a directory listing. Visible files include a password database (incident-support.kdbx), confidential business documents (acquisitions.md, legal.md, announcement_encrypted.md) and several .bak backup files. The path was disclosed in robots.txt.

Impact: Anyone can read internal and credential-related files.

Evidence: Screenshot of the directory listing.

Rating note: Task 1 rated this Medium and Task 3 rated it High. It is rated High because the exposed files include credential-related and confidential material.

Remediation:

- Disable directory listing on the web server.
- Move non-public files outside the web root, or require authentication and authorization.
- Remove backup files from deployed systems.
- Audit all publicly reachable paths before release.

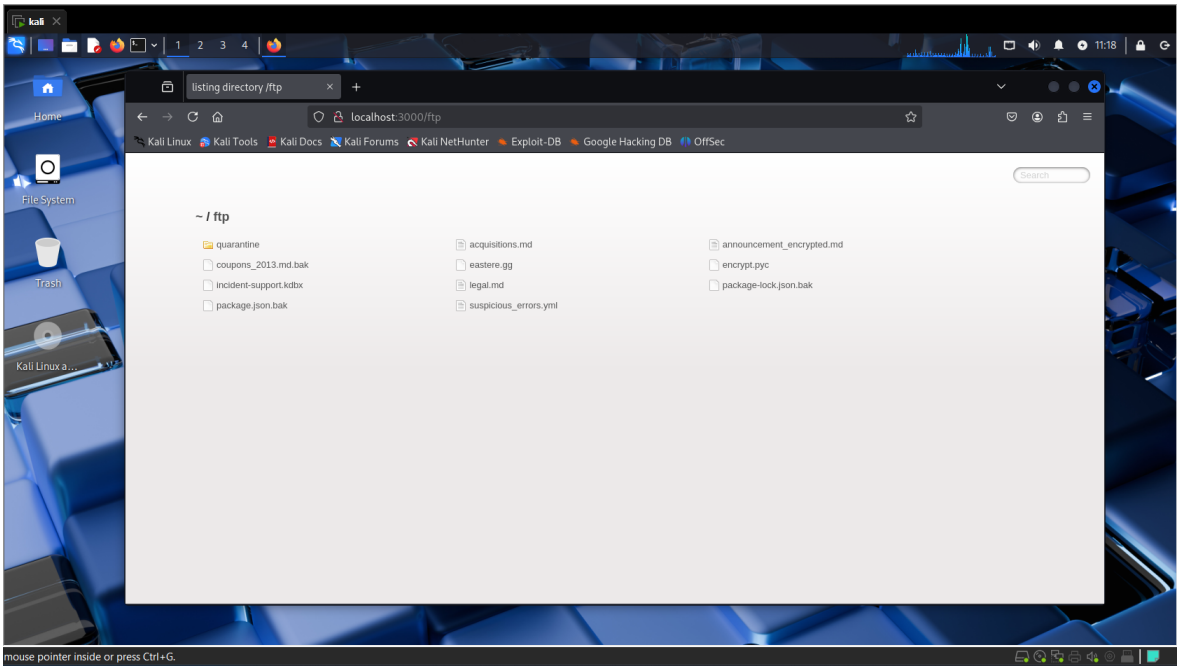


Figure F2-1: Unauthenticated /ftp directory listing

F3: Credentials and Cookies in Cleartext HTTP - High

Affected asset: HTTP login request to Juice Shop (captured on the lab network)

Description: Using the filter `http.request.method == "POST"` and Follow TCP Stream in Wireshark, the login email, password and session cookies were readable in plain text. The password used was a deliberately wrong one.

Impact: Anyone able to observe the network path could steal credentials and hijack sessions.

Evidence: Follow TCP Stream screenshot (password and cookie values redacted).

Remediation:

- Enforce HTTPS everywhere and redirect all HTTP traffic.
- Enable HSTS.
- Set the Secure and HttpOnly flags on session cookies.

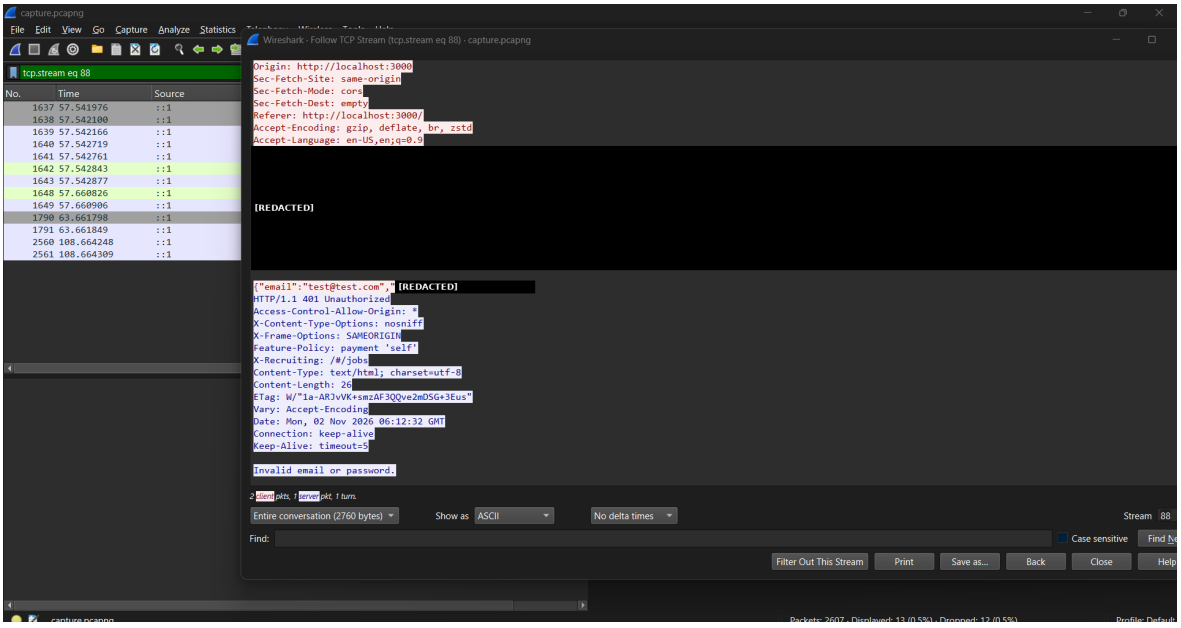


Figure F3-1: Follow TCP Stream (tcp.stream eq 88), sensitive values redacted

F4: Detectable TCP SYN Port Scan Pattern - High

Affected asset: Network monitoring and detection (scan run by me against my own loopback)

Description: The scan sent about 1,000 bare SYN packets from a single source port to different destination ports, roughly 30 microseconds apart, with no completed connections. The filter `tcp.flags.syn == 1 && tcp.flags.ack == 0` exposes the pattern.

Impact: On a real network this pattern indicates reconnaissance ahead of an attack. An unnoticed scan gives an attacker a map of exposed services.

Evidence: Wireshark screenshot of the scan pattern.

Remediation:

- Alert on one source touching many ports in a short time.
- Use an IDS/IPS or firewall rate limits to detect and block scanning.
- Expose only the ports that are required.

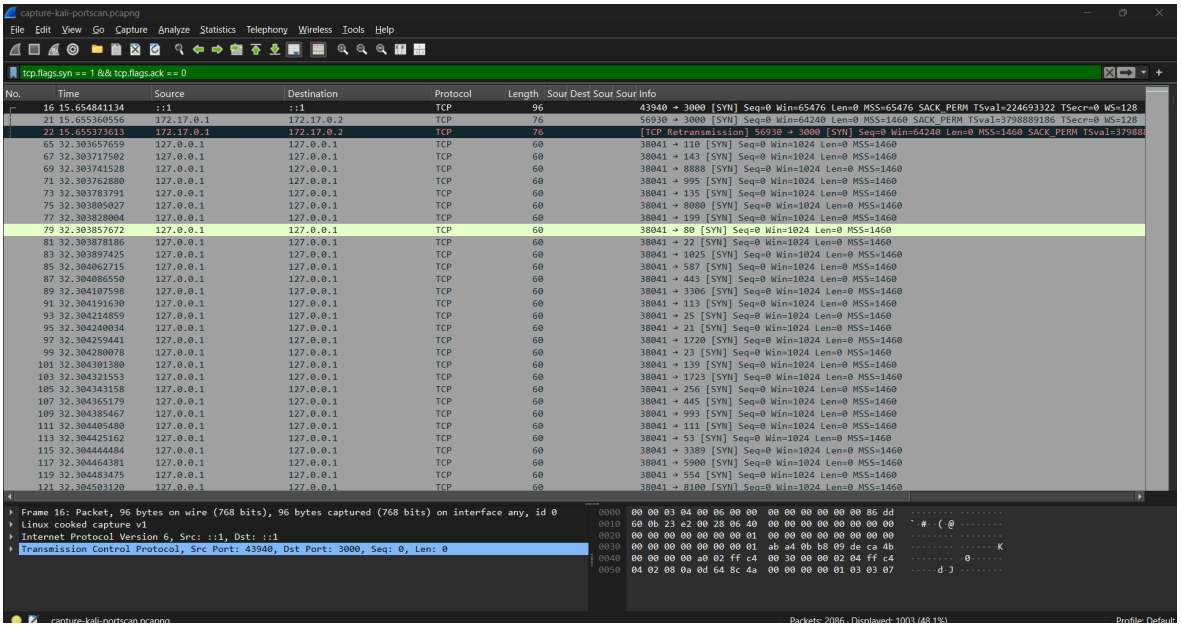


Figure F4-1: SYN-only packets to many destination ports

F5: Cross-Site Scripting in Search - Medium

Affected asset: `GET /#/search?q=`

Description: Search input is rendered into the page without output encoding. The payload `<iframe src="javascript:alert('xss')">` executed JavaScript in the browser.

Impact: Possible session hijacking or phishing when combined with a social-engineering link.

Evidence: Screenshot of the alert popup.

Rating note: Task 1 labeled this DOM-based XSS and Task 3 labeled it reflected. Because the search page is rendered client-side, DOM-based is the more accurate label.

Remediation:

- Encode user-controlled output for its HTML context.
- Deploy a strict Content Security Policy.
- Avoid unsafe DOM sinks such as innerHTML for user input.

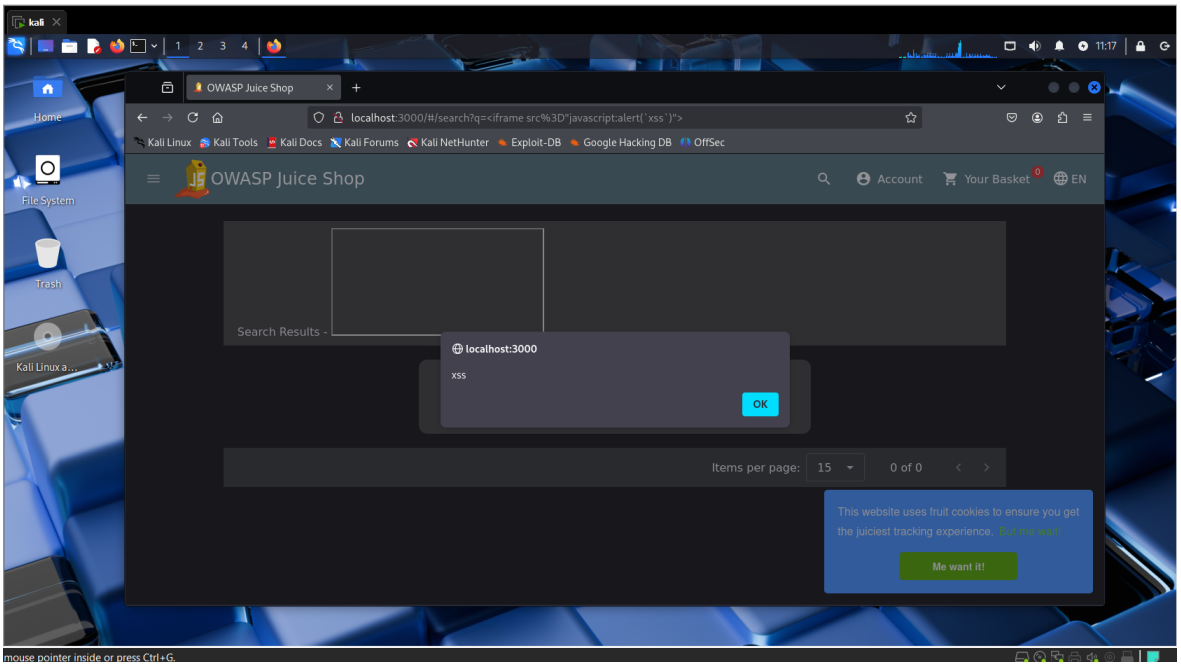


Figure F5-1: JavaScript alert triggered from the search field

F6: Unencrypted DNS and WPAD Lookup - Medium

Affected asset: Host DNS behaviour

Description: The capture held 78 DNS queries for 25 unique names, mostly unencrypted and largely over TCP, plus a WPAD lookup.

Impact: Unencrypted DNS reveals browsing activity. WPAD lookups can be abused to push a malicious proxy configuration.

Evidence: Screenshot of the DNS filter results (addresses redacted).

Remediation:

- Use DNS over HTTPS or DNS over TLS.
- Disable WPAD auto-discovery where it is not needed.
- Monitor DNS for unusually long names and bursts of random sub-domains.

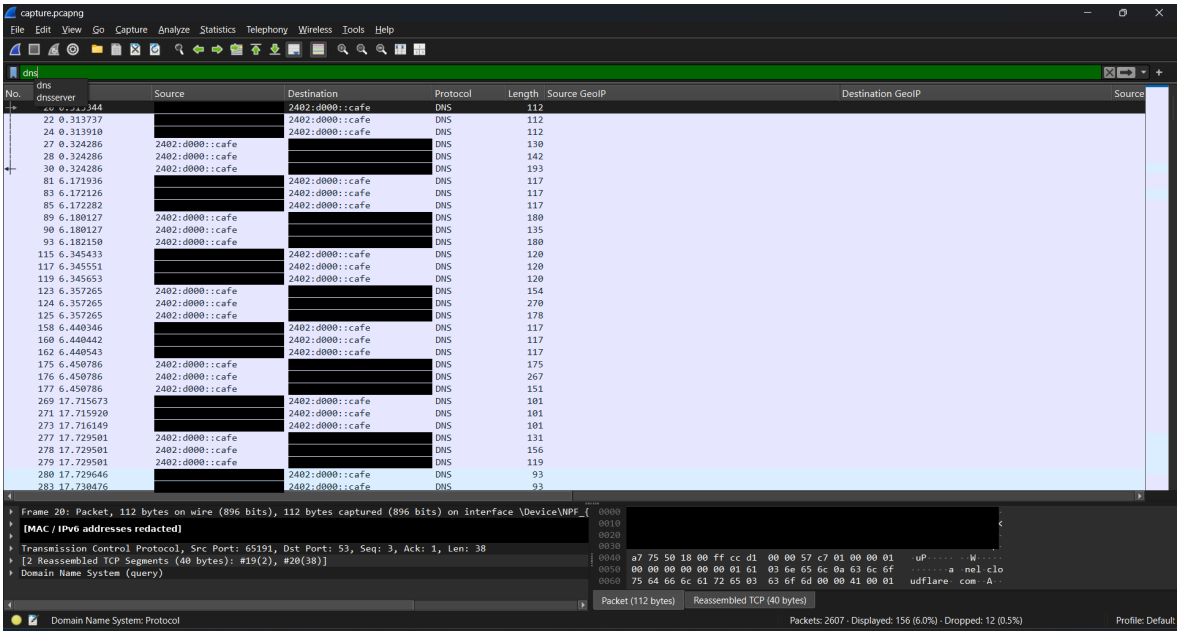


Figure F6-1: DNS queries in the capture (host addresses redacted)

F7: Security Misconfiguration - Low

Affected asset: HTTP response headers and server fingerprint

Description: Reconnaissance found `Access-Control-Allow-Origin: *`, no Content Security Policy or HSTS, a non-standard `X-Recruiting: /#/jobs` header that discloses an internal path, and robots.txt revealing the sensitive /ftp/ path.

Impact: Each item is low impact alone, but together they remove layers of defense and make other attacks easier.

Evidence: Nmap, curl header and Nikto output.

Rating note: Task 1 rated this Low and Task 3 rated it Medium. It is rated Low because no single item gives direct access.

Remediation:

- Restrict CORS to an allow-list of known origins.
- Add CSP and HSTS headers.
- Remove headers that disclose internal details.
- Do not use robots.txt as access control.



Figure F7-1: Nmap service scan of port 3000



Figure F7-2: HTTP response headers and Nikto start

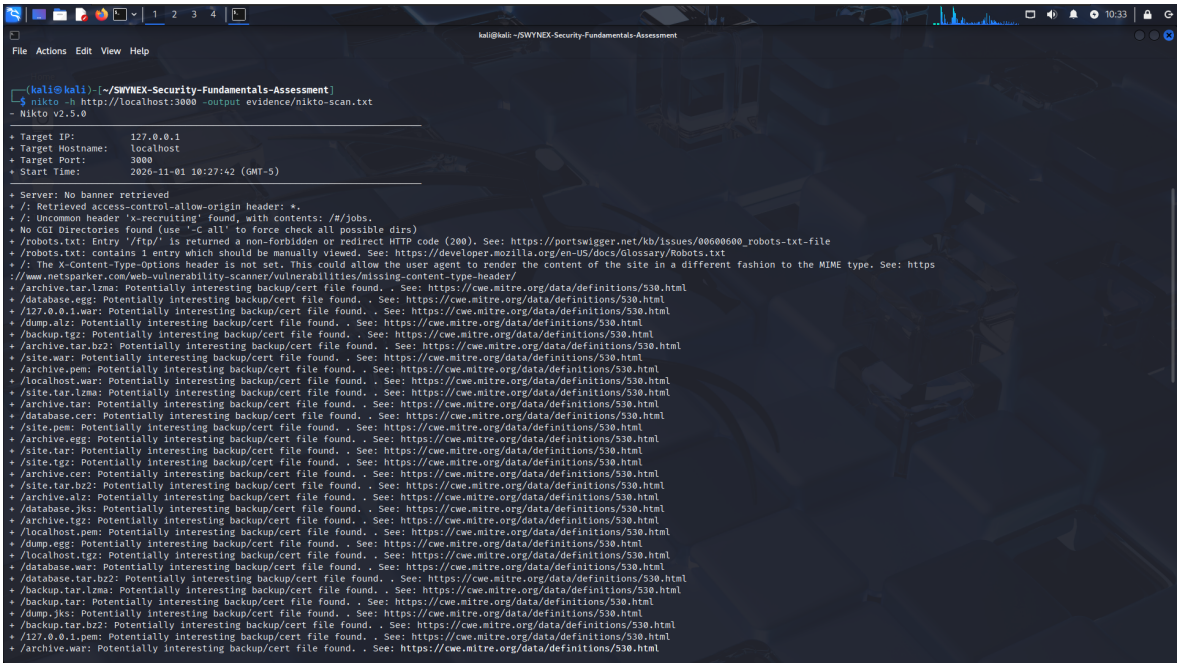


Figure F7-3: Nikto results

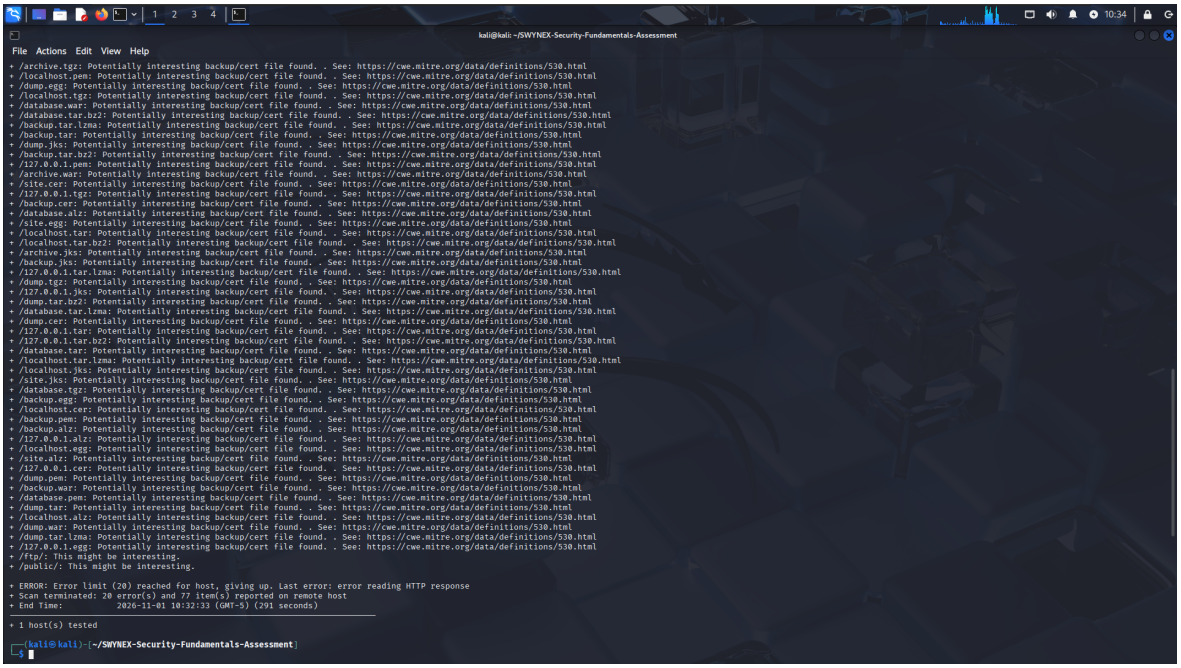


Figure F7-4: Nikto results (repeated backup/cert hits triaged as false positives)

F8: Local SMB and IPv6 Neighbour Discovery Traffic - Informational

Affected asset: Local network services (loopback SMB on port 445, ICMPv6)

Description: SMB/SMB2 made up about 2.8% of packets and ICMPv6 about 0.5%. This is normal background traffic, not an attack.

Impact: None directly, but each protocol is attack surface worth knowing about.

Evidence: Protocol Hierarchy screenshot.

Remediation:

- Disable SMB and IPv6 where they are not needed.

- Restrict SMB to trusted hosts with a firewall.

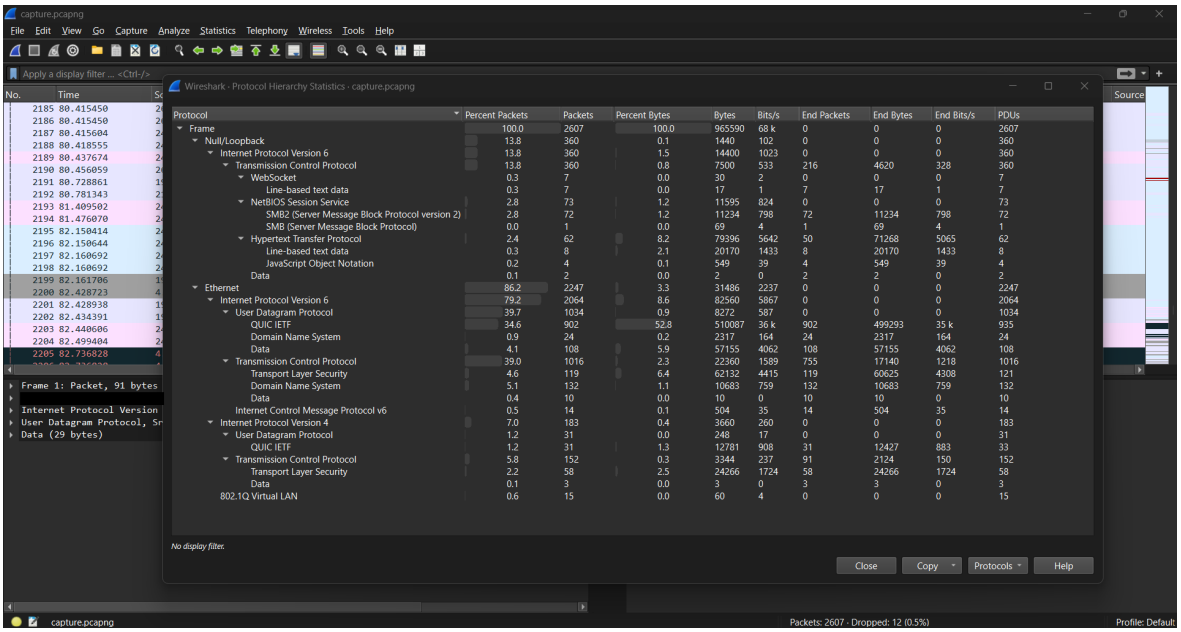


Figure F8-1: Wireshark Protocol Hierarchy for the main capture

7. Remediation Roadmap

Priority	Action	Addresses
1 (immediate)	Parameterized queries for all database access	F1
1 (immediate)	Remove public /ftp/, disable directory listing, delete backup files	F2
2 (short term)	Enforce HTTPS and HSTS, secure cookie flags	F3, F7
2 (short term)	Output encoding and a strict CSP	F5, F7
3 (medium term)	Scan detection alerts and firewall rate limiting	F4
3 (medium term)	Encrypted DNS and WPAD disabled	F6
4 (ongoing)	Restrict CORS, remove disclosing headers, trim unneeded services	F7, F8

8. Limitations

- The target is intentionally vulnerable, so real-world prevalence of these flaws will differ.
- Testing used Community editions of tools and focused on the OWASP Top 10, not a full penetration test.
- Encrypted traffic (about 43% of packets, mainly QUIC and TLS) was counted but not inspected.
- Network findings come from my own lab traffic, so their ratings describe the risk on a real network.

9. Conclusion

The assessments showed how a small number of common flaws (unsanitized input, missing access control and unencrypted traffic) can combine into a complete compromise. They also showed how much a defender can learn from careful traffic analysis and evidence-first documentation. Fixing F1 to F3 first would remove the most severe risks, and the remaining items would then harden the overall security posture.

10. Data Handling Statement

The raw Wireshark capture also contained personal browsing traffic, so it is not published. All passwords, session tokens, cookies, MAC addresses and personal IPv6 addresses were redacted from the screenshots in this report before submission.